

Unit 3: Users and Privileges

- In Oracle databases, users and privileges are essential concepts for managing access control and data security.
- **Users** are accounts that represent individuals or applications that can access the database. Each user has a unique username and can be assigned a password for authentication. Users can be created, dropped, and have their attributes managed by a database administrator (DBA).
- **Privileges** are permissions granted to users that control what actions they can perform on database objects. Privileges define the level of access a user has to specific objects or operations in the database. Privileges can be granted, revoked, and modified by a DBA
- Understanding users and privileges is crucial for securing your Oracle database and ensuring that users have the appropriate access to perform their tasks while protecting sensitive data.

Managing Users

- To connect to the database, each user must specify a valid user name that has been previously defined to the database. An account must have been established for the user, with information about the user being stored in the data dictionary.

Predefined Database Administration Accounts

- Oracle Database includes several predefined user accounts. The three types of predefined accounts are:
 - **Administrative accounts**
 - **Sample schema accounts**
 - **Internal accounts**

Administrative Accounts

- Administrative user accounts have special privileges required to administer areas of the database, such as the CREATE ANY TABLE or ALTER SESSION privilege, or EXECUTE privilege on packages owned by the SYS schema.
- The following administrative user accounts are automatically created when Oracle Database is installed:

- SYS
- SYSTEM
- SYSBACKUP
- SYSDG
- SYSKM
- SYSRAC
- SYSMAN
- DBSNMP
- You must not delete these accounts.
- Oracle recommends that you create at least one additional administrative user and grant it appropriate privileges for performing daily administrative tasks

Sample Schema Accounts

- These optional accounts are used for examples in Oracle Database documentation and instructional materials. The sample schema accounts are – HR, SH, and OE.
- The sample database schemas provide a common platform for examples in each release of the Oracle Database. The sample schemas are a set of interlinked database schemas. This set provides approach to complexity:
 - *Schema Human Resources (HR)* is useful for introducing basic topics. An extension to this schema supports Oracle Internet Directory demos.
 - *Schema Order Entry (OE)* is useful for dealing with matters of intermediate complexity. Many data types are available in this schema, including nonscalar data types.
 - *Schema Online Catalog (OC)* is a collection of object-relational database objects built inside schema OE.
 - *Schema Product Media (PM)* is dedicated to print media data types.
 - A set of schemas gathered under the main schema *name Information Exchange (IX)* can be used to demonstrate Oracle Advanced Queuing capabilities.
 - *Schema Sales History (SH)* is designed to allow for demos with large amounts of data. An extension to this schema provides support for advanced analytic processing.
 - *Schema Customer Orders (CO)* is a modern schema useful for demos of e-commerce transactions. It allows the storage of semi-structured data using JSON.

- The Oracle Database sample schemas are based on a fictitious sample company that sells goods through various channels. The company operates worldwide to fill orders for products. It has several divisions, each of which is represented by a sample database schema.
 - **Schema HR** – Division Human Resources tracks information about the company employees and facilities.
 - **Schema OE** – Division Order Entry tracks product inventories and sales of company products through various channels.
 - **Schema PM** – Division Product Media maintains descriptions and detailed information about each product sold by the company.
 - **Schema IX** – Division Information Exchange manages shipping through B2B applications.
 - **Schema SH** – Division Sales tracks business statistics to facilitate business decisions.
 - **Schema CO** - Division Customer Orders models a simple retail application consisting of customer, product, store and order data.
- Each sample schema has a user account associated with it. For example, the hr user account owns the hr schema, which contains a set of simple tables for a human resources application. The sample schema accounts are also initially locked and have an expired password. As the database administrator, you are responsible for unlocking these accounts and assigning passwords to these accounts.

Internal accounts

- All databases also include internal accounts, which are automatically created so that individual Oracle Database features or components such as Oracle Application Express can have their own schemas. To protect these accounts from unauthorized access, they are initially locked and their passwords are expired. (A locked account is an account for which login is disabled.) You must not delete internal accounts, and you must not use them to log in to the database.
- These accounts are created so that individual Oracle Database features or components can have their own schemas. You must not delete internal accounts, and you must not attempt to log in with them. Internal accounts are used by Oracle Database for specific tasks and might not be directly manageable.

- Certain user accounts are created automatically for database administration. Examples are SYS and SYSTEM. Other accounts are automatically created just so that individual Oracle Database XE features or products can have their own schemas. An example is the CTXSYS account, which is used by the Oracle Text product. Oracle Text is used to index the Oracle Database XE online Help. The Help index is stored in the CTXSYS schema in the database. These automatically created accounts are called internal user accounts, and their schemas are called internal schemas.

User Accounts

- For users to access your database, you must create user accounts and grant appropriate database access privileges to those accounts. A user account is identified by a user name and defines the attributes of the user, including the following:
 - Authentication method
 - Password for database authentication
 - Default tablespaces for permanent and temporary data storage
 - Tablespace quotas
 - Account status (locked or unlocked)
 - Password status (expired or not)
- When you create a user account, you must not only assign a user name, a password, and default tablespaces for the account, but you must also do the following:
 - Grant the appropriate system privileges, object privileges, and roles to the account.
 - If the user will be creating database objects, then give the user account a space usage quota on each tablespace in which the objects will be created.
- Oracle recommends that you grant each user just enough privileges to perform his job, and no more. For example, a database application developer needs privileges to create and modify tables, indexes, views, and stored procedures, but does not need (and should not be granted) privileges to drop (delete) tablespaces or recover the database. You can create user accounts for database administration, and grant only a subset of administrative privileges to those accounts.

- In addition, you may want to create user accounts that are used by applications only. That is, nobody logs in with these accounts; instead, applications use these accounts to connect to the database, and users log in to the applications. This type of user account avoids giving application users the ability to log in to the database directly, where they could unintentionally cause damage.
- When you create a user account, you are also implicitly creating a schema for that user. A schema is a logical container for the database objects (such as tables, views, triggers, and so on) that the user creates. The schema name is the same as the user name, and can be used to unambiguously refer to objects owned by the user. For example, hr. employees refers to the table named employees in the hr schema. (The employees table is owned by hr.) The terms database object and schema object are used interchangeably.
- When you delete a user, you must either simultaneously delete all schema objects of that user, or you must have previously deleted the schema objects in separate operations.

Creating User Accounts

- A user account can have restrictions such as profiles, a default role, and tablespace restrictions.
- In a multitenant environment, CDB common users and application common have access to their respective containers, and local users are specific to a PDB.
- Oracle provides two types of common users: CDB common users and application common users.
- In a multitenant environment, a local user is a database user that exists only in a single PDB

Who Can Create User Accounts?

- Users who has been granted the CREATE USER system privilege can create user accounts, including user accounts to be used as proxy users.
- Because the CREATE USER system privilege is a powerful privilege, a database administrator or security administrator is usually the only user who has this system privilege.

- If you want to create users who themselves have the privilege to create users, then include the `WITH ADMIN OPTION` clause in the `GRANT` statement. For example:

```
GRANT CREATE USER TO lbrown WITH ADMIN OPTION;
```

- In a multitenant environment, you must have the commonly granted `CREATE USER` system privilege to create common user accounts. To create local user accounts, you must have a commonly granted `CREATE USER` privilege or a locally granted `CREATE USER` privilege in the PDB in which the local user account will be created.

Creating a New User Account That Has Minimum Database Privileges

When you create a new user account, you should enable this user to access the database

- Use the `CREATE USER` statement to create a new user account.

The basic syntax of the `CREATE USER` statement is as follows:

```
CREATE USER username  
  
IDENTIFIED BY password  
  
[DEFAULT TABLESPACE tablespace]  
  
[QUOTA {size | UNLIMITED} ON tablespace]  
  
[PROFILE profile]  
  
[PASSWORD EXPIRE]  
  
[ACCOUNT {LOCK | UNLOCK}];
```

In this syntax:

CREATE USER username

Specify the name of the user to be created.

IDENTIFIED BY password

Specify a password for the local user to use to log on to the database. Note that you can create an external or global user, which is not covered here.

DEFAULT TABLESPACE

Specify the tablespace of the objects such as tables and views that the user will create.

If you skip this clause, the user's objects will be stored in the database default tablespace if available, typically it is USERS tablespace; or the SYSTEM tablespace in case there is no database default tablespace.

QUOTA

Specify the maximum space in the tablespace that the user can use. You can have multiple QUOTA clauses, each for a tablespace.

Use UNLIMITED if you don't want to restrict the size of the tablespace that the user can use.

PROFILE profile

A user profile limits the database resources or password that the user cannot exceed. You can assign a profile to a newly created user. If you skip this clause, Oracle will assign the DEFAULT profile to the user.

PASSWORD EXPIRE

Use the PASSWORD EXPIRE if you want to force the user to change the password for the first time the user logs in to the database.

ACCOUNT {LOCK | UNLOCK}

Use ACCOUNT LOCK if you want to lock the user and disable access. On the other hand, specify ACCOUNT UNLOCK to unlock user and enable access.

NOTE: To execute the CREATE USER statement, you must have the CREATE USER system privilege. Once you create the new user, the privilege domain of the user will

be empty. Therefore, if you want the user to be able to login to the database, you should grant the CREATE SESSION system privilege to the user.

Examples:

- Using Oracle CREATE USER statement to create a new local user example

```
CREATE USER bipin IDENTIFIED BY abcd1234;
```

This example uses the CREATE USER statement to create a new local user named bipin with the password abcd1234:

- NOTE: To enable the user john to log in, you need to grant the CREATE SESSION system privilege to the user bipin by using the following statement:

```
GRANT CREATE SESSION TO bipin;
```

- Another example:

```
CREATE USER new_user
IDENTIFIED BY secure_password
DEFAULT TABLESPACE example
QUOTA 50M ON example
PROFILE my_profile
PASSWORD EXPIRE;
```

This command creates a new user named new_user with the following properties:

- **Password:** The user's password is set to "secure_password". Remember to replace this with a strong and unique password in your actual use case.
- **Default Tablespace:** The user's default tablespace is set to example. Make sure this tablespace already exists in your database.
- **Quota:** The user is granted a quota of 50 megabytes (50M) on the example tablespace. This limits the amount of space the user can use for their permanent data objects.

- **Profile:** The user is assigned a profile named my_profile. This profile can define specific settings and limitations for the user, such as resource consumption or password complexity rules.
- **Password Expiration:** The user's password is set to expire after a certain period (defined by a database parameter). This enforces regular password changes for improved security.

User Authentication

- Authentication means to verify the identity of users or other entities that connect to the database.
- Authentication is the way of verifying the identity of a user, device, or other entity who wants to use data, resources, or applications.
- Validating this identity establishes a trust relationship for further interactions. Authentication also enables accountability by making it possible to link access and actions to specific identities. After authentication, authorization processes can allow or limit the levels of access and action permitted to that entity.
- You can authenticate database administrators by using strong authentication, from the operating system, or from the database using passwords.
- You can secure user passwords in a variety of ways, such as controlling the password creation requirements or using password management policies.
- Oracle Database can authenticate by using information that is maintained by the operating system.
- You can authenticate users over a network by using Transport Layer Security with third-party services.

What Are the Oracle Database Built-in Password Protections?

Oracle Database provides a set of built-in password protections designed to protect your users' passwords.

These password protections are as follows:

- **Password encryption:** Oracle Database automatically and transparently encrypts passwords during network (client-to-server and server-to-server) connections, using

Advanced Encryption Standard (AES) before sending them across the network. However, a password that is specified within a SQL statement (such as `CREATE USER user_name IDENTIFIED BY password;`) is still transmitted across the network in clear text in the network trace files. For this reason, you should have native network encryption enabled or configure Transport Layer Security (TLS) encryption.

- **Password complexity checking:** In a default installation, Oracle Database provides the `ora12c_verify_function` and `ora12c_strong_verify_function` password verification functions to ensure that new or changed passwords are sufficiently complex to prevent intruders who try to break into the system by guessing passwords. You must manually enable password complexity checking. You can further customize the complexity of your users' passwords.
- **Preventing passwords from being broken.** If a user tries to log in to Oracle Database multiple times using an incorrect password, Oracle Database delays each login by one second. This protection applies for attempts made from different IP addresses or multiple client connections. This feature significantly decreases the number of passwords that an intruder would be able to try within a fixed time period when attempting to log in. The failed login delay slows down each failed login attempt, increasing the overall time that is required to perform a password-guessing attack, because such attacks usually require a very large number of failed login attempts.

For non-administrative logins, Oracle Database protects against concurrent password guessing attacks by setting an exclusive lock for the failed login delay. This prevents an intruder from attempting to sidestep the failed login delay when the intruder tries the next concurrent guess in a different database session as soon as the first guess fails and is delayed.

By holding an exclusive lock on the account that is being attacked, Oracle Database mitigates concurrent password guessing attacks, but this can simultaneously leave the account vulnerable to denial-of-service (DoS) attacks. To remedy this problem, you should create a password profile where the `FAILED_LOGIN_ATTEMPTS` parameter is set to `UNLIMITED`, and then apply this password profile to the user account. The value `UNLIMITED` for the `FAILED_LOGIN_ATTEMPTS` parameter setting disables failed login delays

and does not limit the number of failed login attempts. For these types of accounts, Oracle recommends that you use a long random password.

The concurrent password-guessing attack protection does not apply to administrative user connections, because these kinds of connections must remain available at all times and be immune to denial-of-service attacks. Hence, Oracle recommends that you choose long passwords for any administrative privileged account.

- **Enforced case sensitivity for passwords:** Passwords are case sensitive. For example, the password hPP5620qr fails if it is entered as hpp5620QR or hPp5620Qr. Case sensitivity affects password files and database links.
- **Passwords hashed using the 12C password version.** To verify the user's password and enforce case sensitivity in password creation, Oracle Database uses the 12C password version, which is based on a de-optimized algorithm that involves Password-Based Key Derivation Function (PBKDF2) and the SHA-512 cryptographic hash functions.

Minimum Requirements for Passwords

- Oracle provides a set of minimum requirements for passwords.
- Passwords can be at most 30 bytes long. There are a variety of ways that you can secure passwords, ranging from requiring passwords to be of a sensible length to creating custom password complexity verification scripts that enforce the password complexity policy requirements that apply at your site.

Creating a Password by Using the IDENTIFIED BY Clause

SQL statements that accept the IDENTIFIED BY clause also enable you to create passwords.

- To create passwords for users, use the CREATE USER, ALTER USER, GRANT CREATE SESSION, or CREATE DATABASE LINK SQL statement.
- The following SQL statements create passwords with the IDENTIFIED BY clause.

```
CREATE USER psmith IDENTIFIED BY password;
GRANT CREATE SESSION TO psmith IDENTIFIED BY password;
ALTER USER psmith IDENTIFIED BY password;
CREATE DATABASE LINK AUTHENTICATED BY psmith IDENTIFIED BY
password;
```

Using a Password Management Policy

- A password management policy can create and enforce a set of restrictions that can better secure user passwords.
- Database security systems that depend on passwords require that passwords be kept secret at all times.
- Because passwords are vulnerable to theft and misuse, Oracle Database uses a **password management policy**. Database administrators and security officers control this policy through **user profiles**, enabling greater control of database security.
- You can use the CREATE PROFILE statement to create a user profile. The profile is assigned to a user with the CREATE USER or ALTER USER statement.

Changing a User's Password

- The ALTER USER statement allows you to change the password for a user, including yourself, depending on your privileges. Here's the syntax:
ALTER USER username IDENTIFIED BY new_password;
- To change another user's password, you need to have a privileged account like SYS or SYSTEM.
- You can change your own password if you are logged in as that user.
- You can change your own password by first connecting to the system with your credential and executing the ALTER USER statement like below

```
ALTER USER PASSWORD new_secure_password;
```

NOTE: Some Oracle tools offer graphical interfaces for user management, including password changes.

Managing a User Account

You can use GUI tools like SQL Developer or the SQL Command Line (SQL*Plus) to manage database users. User management refers to the activities like

- Creating Users
- Altering Users
- Locking and Unlocking User Accounts
- Expiring a User Password
- Dropping Users

Creating Users:

- This involves defining a new user account in the database with a username, password, and potentially other attributes like default tablespace, quota limitations, and assigned profile.
- The CREATE USER statement in SQL is used for this purpose.

Altering Users:

This encompasses modifying existing user accounts. This might include:

- Changing the user's password using ALTER USER statement or client tools.
- Adjusting quota limitations on tablespaces.
- Modifying the default tablespace assigned to the user.
- Assigning a different profile that defines user permissions.
- Granting or revoking specific privileges to the user.

Locking and Unlocking User Accounts:

- Locking a user account prevents the user from logging in to the database. This is useful for security reasons, such as when a password is suspected to be compromised or the user needs to be temporarily disabled.
- Unlocking a user account allows a previously locked user to regain access using their valid credentials.

- The ALTER USER statement with the ACCOUNT LOCK or ACCOUNT UNLOCK clause can be used for this purpose.

Expiring a User Password:

- This forces the user to change their password upon their next login attempt. This encourages regular password updates for improved security.
- The PASSWORD EXPIRE clause can be added to the CREATE USER statement or used with ALTER USER to enable password expiration for an existing user.

Dropping Users:

- This permanently removes a user account from the database. This action should be done with caution as it removes all user-owned objects and privileges associated with the account.
- The DROP USER statement is used to delete a user account.
- A user who is connected to a database cannot be dropped. You must first terminate the user session (or the user can exit the session) before you can drop the user.

Benefits of User Management:

- *Improved Security:* Allows for granular control over who can access the database and the type of access they have.
- *Data Integrity:* Ensures that only authorized users can perform actions on data, minimizing the risk of unauthorized modifications.
- *Manageability:* Simplifies administration by allowing control over different user groups and their access levels.

Dropping user Account

- Use the DROP USER statement to remove a database user and optionally remove the user's objects.

- In an Oracle Automatic Storage Management (Oracle ASM) cluster, a user authenticated AS SYSASM can use this clause to remove a user from the password file that is local to the Oracle ASM instance of the current node.
- When you drop a user, Oracle Database also purges all of that user's schema objects from the recycle bin.
- You must have the DROP USER system privilege. In an Oracle ASM cluster, you must be authenticated AS SYSASM.

DROP USER {username} [CASCADE];

Explanation:

DROP USER: This keyword initiates the command to remove a user from the database.

{username}: Specifies the user account you want to delete.

[CASCADE]: If included, this clause instructs Oracle to automatically drop all database objects owned by the user before deleting the user account itself. This includes tables, views, indexes, sequences, etc.

Important Notes:

- Be cautious: Dropping a user is permanent and cannot be undone. Ensure you have a backup and understand the implications before proceeding.
- Use CASCADE with care: If the user owns many objects, cascading the drop can take time and might require additional privileges depending on the objects involved.
- Check for dependencies: If other users or applications rely on objects owned by the user you want to drop, dropping the user might cause errors. You'll need to address these dependencies first, such as transferring ownership of objects or dropping them explicitly.
- Privileges: To drop a user, you need the DROP USER system privilege. This privilege is typically granted to administrative accounts like SYS or SYSTEM.

Example (without CASCADE):

```
DROP USER old_user;
```

This will attempt to drop the user `old_user`. If the user owns any objects, the statement will fail, and you'll need to handle those objects before dropping the user.

Example (with CASCADE):

```
DROP USER inactive_user CASCADE;
```

This will drop the user `inactive_user` and automatically delete all objects owned by that user.

Notes:

- Always back up your database before making significant changes like dropping users.
- Consider using tools like SQL Developer for a more user-friendly interface for dropping users.

Monitor User Information

There are several ways to monitor user information in Oracle. Some of them are as follows

❖ **Using Views:** Oracle provides several views that contain information about users and their activity. Here are a few commonly used ones:

- **DBA_USERS:** This view shows basic information about all users in the database, including username, account status (locked/unlocked), default tablespace, and quota information.
- **DBA_USER_ROLES:** This view displays the roles assigned to each user.
- **DBA_ROLE_PRIVILEGES:** This view lists the privileges granted to each role in the database.
- **V\$SESSION:** This view provides real-time information about active database sessions, including the username associated with each session, the status of the session, and the SQL statement currently being executed (if any).

You can query these views using SQL*Plus or other SQL clients to retrieve user information. Here's an example:

[This query will display the username, account status, and default tablespace for all users.]

```
SELECT username, account_status, default_tablespace
FROM dba_users;
```

❖ **Using Performance Monitoring Tools:**

- Oracle Database offers built-in performance monitoring tools like Automatic Workload Repository (AWR) and Automatic Database Diagnostic Miner (ADDM). These tools can provide insights into user activity and resource consumption.
- For instance, you can identify frequently used queries by specific users or analyse overall database workload associated with different users.

❖ **Using Oracle Database Enterprise Manager (OEM):**

- If you're using Oracle Database with Enterprise Manager, it provides a graphical user interface for managing and monitoring users. You can view user details, assigned roles and privileges, and track user activity through reports and dashboards.

❖ **Auditing User Actions:**

- Oracle Database Auditing allows you to track specific user actions within the database. You can define audit policies to log events like successful/failed login attempts, DML (data manipulation language) statements executed by users, and other activities. By analysing audit logs, you can monitor user behaviour and identify potential security risks.

About User Sessions

- A **session** refers to the connection established by a user process (client application) to the Oracle Database instance. Each session represents a single interaction between a user and the database. Sessions are fundamental to Oracle's client-server architecture and play a crucial role in database operations.

Session Lifespan:

- The lifespan of a session in Oracle can vary based on factors such as user activity, application design, and database configuration. Sessions typically remain active as long as the corresponding user process maintains the connection to the database. However, sessions may be terminated due to user-initiated disconnections, administrative actions, or system failures.

Session Information:

Oracle provides various views to access information about active sessions. The most commonly used view is V\$SESSION. Here are some key details stored about sessions in this view:

- Username: The user associated with the session.
- SID (Session ID): A unique identifier for the session.
- Serial Number (SERIAL#): An additional identifier for the session within the SID.
- Status: Indicates the current state of the session (e.g., ACTIVE, INACTIVE, WAITING).
- SQL Statement (if applicable): The SQL statement currently being executed by the session (if any).
- Other information: Additional details like resource usage, wait events, and connection details.

Session Lifecycle:

Establishment: When a user or application connects to the database using a username and password, a new session is established. The database assigns a unique session identifier (SID) and serial number to the session.

Activity: During the session, the user or application can execute SQL statements, retrieve data, and perform other database operations.

Termination: The session ends when the user disconnects, encounters an error, remains inactive for a certain period (defined by database parameters), or is terminated by an administrator.

Importance of Sessions:

Some importance are as follows

- *User Interaction:* Sessions are fundamental for users and applications to interact with the database and perform operations.
- *Performance Monitoring:* By analyzing session information, you can monitor database activity, identify potential bottlenecks, and optimize performance.
- *Security:* Session management allows for controlling user access and resource consumption within the database.

Managing Sessions:

Oracle provides various mechanisms for managing sessions:

- *Automatic Termination:* Inactive sessions can be automatically terminated after a predefined idle time.
- *Forced Termination:* Administrators can forcefully terminate sessions using commands like `ALTER SYSTEM KILL SESSION` or `ALTER SYSTEM DISCONNECT SESSION`
- *Monitoring Tools:* Performance monitoring tools in Oracle Database can help to track session activity and identify resource-intensive sessions.

Terminate User Sessions

A user who is connected to a database cannot be dropped. You must first terminate the user session (or the user can exit the session) before you can drop the user.

We can use following methods to terminate user sessions in Oracle 19c by using SqlPlus

❖ Using the ALTER SYSTEM KILL SESSION statement:

- ☞ This method allows you to terminate a specific user session by providing the session's sid (session identification number) and serial# (serial number). Here's the syntax:

```
ALTER SYSTEM KILL SESSION '<sid>,<serial#>' [IMMEDIATE];
```

Explanation of the syntax:

☞ `<sid>,<serial#>`: Replace this with the comma-separated sid and serial# of the session you want to terminate.

☞ `[IMMEDIATE]`: This is an optional clause.

— By default, the KILL SESSION option sends a termination request to the session. The session might have the opportunity to complete its current operation before terminating.

— Adding the IMMEDIATE clause forcefully terminates the session without waiting for any ongoing operation to finish. This can lead to data loss or transaction rollbacks if the user was modifying data when the session was terminated.

❖ Using the ALTER SYSTEM DISCONNECT SESSION statement:

This method allows you to disconnect a user session in a more controlled manner. Here's the syntax:

```
ALTER SYSTEM DISCONNECT SESSION '<sid>,<serial#>'
[POST_TRANSACTION | IMMEDIATE];
```

Explanation of the syntax:

☞ `<sid>,<serial#>`: Replace this with the comma-separated sid and serial# of the session you want to disconnect.

☞ `[POST_TRANSACTION | IMMEDIATE]`: This clause specifies when the session is disconnected.

— POST_TRANSACTION: Allows the session to complete the current transaction before disconnecting. This is the default behavior if you don't specify a clause.

— IMMEDIATE: Disconnects the session immediately, potentially rolling back any uncommitted transactions.

NOTE: Before terminating sessions, you'll need to identify the specific sessions you want to target. Use the `V$SESSION` view to retrieve information about active sessions, including username, sid, serial#, status, and other details.

Schema Only Account

- You can create schema-only accounts, that is, the schema user has no password.
- Schema Only Account offer a secure way to manage database objects without the associated user having the ability to directly log in
- This type of account, designed for some Oracle-provided schemas along with some user-created schemas, can be created without the specification of a password or an authentication type. It cannot be authenticated unless an authentication method is assigned by using the `ALTER USER` statement
- By default, most of the predefined schema user accounts that are available with Oracle Database, such as the sample schema user accounts (for example, HR), are schema-only accounts. You can assign these accounts passwords if you want to, but for better security, Oracle recommends that you set them back to being schema-only afterwards. To check if a schema user account is schema only, query the `AUTHENTICATION_TYPE` column of the `DBA_USERS` data dictionary view. `NONE` indicates that the account is schema only.
- Note the following rules about using schema only accounts:
 - ☞ Schema only accounts can be used for both administrator and non-administrator accounts.
 - ☞ Schema only accounts must be created on the database instance only, not in Oracle Automatic Storage Management (ASM) environments.
 - ☞ You can grant system privileges (such as `CREATE ANY TABLE`) and administrator roles (such as `DBA`) to schema only accounts. Schema only accounts can create objects such as tables or procedures, assuming they have had to correct privileges granted to them.
 - ☞ You can configure schema only accounts to be used as client users in a proxy authentication in a single session proxy. This is because in a single session proxy, only the credentials of the proxy user are verified, not the credentials of the client user. Therefore, a schema only account can be a client user. However, you cannot configure schema only accounts for a two-proxy

scenario, because the client credentials must be verified. Hence, the authentication for a schema only account will fail.

☞ Schema only accounts cannot connect through database links, either with connected user links, fixed user links, or current user links.

- Schema-only accounts are a valuable security feature for managing sensitive data in Oracle databases. They provide a secure way to control object ownership while allowing controlled access through authorized proxy users. Remember, there's no restriction on the number of sessions a proxy user can establish to access the schema-only account's objects.

Creating a Schema-Only Account

- The **CREATE USER** SQL statement creates schema-only accounts. You can run the **CREATE USER** statement with the **NO AUTHENTICATION** clause only on a database instance. You cannot run it on an Oracle Automatic Storage Management (ASM) instance.

- Use the **CREATE USER** statement with the **NO AUTHENTICATION** clause.

For example:

```
CREATE USER psmith NO AUTHENTICATION;
```

Altering a Schema-Only Account

- The **ALTER USER** SQL statement can be used to modify schema-only accounts.
 1. Check if the schema user has administrative privileges.
You can query the **V\$PWFILE_USERS** to find if the schema user has administrative privileges.
 2. If the schema user has administrative privileges, then use the **REVOKE** statement to revoke these privileges.
 3. Use the **ALTER USER** SQL statement with the **NO AUTHENTICATION** clause to modify the schema account to have no authentication.

For example:

```
ALTER USER psmith NO AUTHENTICATION;
```

Manging Privileges

- A user privilege is the right to run a particular type of SQL statement, or the right to access an object that belongs to another user, run a PL/SQL package, and so on. The types of privileges are defined by Oracle Database.
- Privileges control what users can and cannot do

System privileges

- ☞ These privileges allow the grantee to perform standard administrator tasks in the database. Restrict them only to trusted users.
- ☞ A system privilege is the right to perform an action or to perform actions on schema objects.
- ☞ For example, the privileges to create tablespaces and to delete the rows of any table in a database are system privileges.
- ☞ There are over 100 distinct system privileges. Each system privilege allows a user to perform a particular database operation or class of database operations.
- ☞ System privileges are very powerful. Only grant them when necessary to roles and trusted users of the database.
- ☞ To find the system privileges that have been granted to a user, you can query the DBA_SYS_PRIVS data dictionary view.
- ☞ Only two types of users can grant system privileges to other users or revoke those privileges from them. These users are as follows:
 - Users who were granted a specific system privilege with the ADMIN OPTION
 - Users with the system privilege GRANT ANY PRIVILEGE
- ☞ Applicable to the entire database or server.
- ☞ Some examples:
 - CREATE SESSION
 - CREATE TABLE
 - CREATE SEQUENCE
 - CREATE VIEW
 - CREATE PROCEDURE

Object privileges

- ☞ Each type of object has privileges associated with it. Object privileges enable you to perform actions on schema objects, such as tables or indexes

- ☞ An object privilege grants permission to perform a particular action on a specific schema object.
- ☞ Different object privileges are available for different types of schema objects. The privilege to delete rows from the departments table is an example of an object privilege.
- ☞ Some schema objects, such as clusters, indexes, triggers, and database links, do not have associated object privileges. Their use is controlled with system privileges. For example, to alter a cluster, a user must own the cluster or have the ALTER ANY CLUSTER system privilege.
- ☞ A user automatically has all object privileges for schema objects contained in his or her schema.
- ☞ A user with the GRANT ANY OBJECT PRIVILEGE system privilege can grant any specified object privilege to another user with or without the WITH GRANT OPTION clause of the GRANT statement. A user with the GRANT ANY OBJECT PRIVILEGE privilege can also use that privilege to revoke any object privilege that was granted either by the object owner or by some other user with the GRANT ANY OBJECT PRIVILEGE privilege.
- ☞ Applicable to specific database objects like tables, views, procedures, etc.
- ☞ Control what users can do with those objects.
- ☞ Examples include allowing users to insert data into a table, update existing data, or delete records.
- ☞ Granted to users based on their needs and the security requirements of the data.
- ☞ Some examples
 - SELECT
 - INSERT
 - UPDATE
 - DELETE
 - REFERENCES
 - GRANT
 - EXECUTE

Note:

- ☞ System privileges control high-level actions at the database level, such as managing users and system resources, while object privileges control access to specific database objects and the actions that can be performed on them. Both types of privileges are essential for ensuring security and controlling access to data and functionality within the Oracle Database.

Roles

- ☞ A Role is a collection of privileges and other roles.
- ☞ Some system-defined roles exist, but most are created by administrators.
- ☞ Roles group together privileges and other roles, which facilitates the granting of multiple privileges and roles to users.
- ☞ A **role** groups several privileges and roles, so that they can be granted to and revoked from users simultaneously. You must enable the role for a user before the user can use it.
- ☞ In a multitenant environment, database roles can be specific to a PDB or used throughout the entire system container or application container. A **common role** is a role that is created in the root a **local role** is created in a PDB.
- ☞ A user role is a named collection of privileges that you can create and assign to other users.

The Functionality of Roles

Roles are useful for quickly and easily granting permissions to users. Roles have the following functionality:

- ☞ A role can be granted system or object privileges.
- ☞ Any role can be granted to any database user.
- ☞ Each role granted to a user is, at a given time, either enabled or disabled. A user's security domain includes the privileges of all roles currently enabled for the user and excludes the privileges of any roles currently disabled for the user. Oracle

Database allows database applications and users to enable and disable roles to provide selective availability of privileges.

- ☞ A role can be granted to other roles. However, a role cannot be granted to itself and cannot be granted circularly. For example, role role1 cannot be granted to role role2 if role role2 has previously been granted to role role1.
- ☞ If a role is not password authenticated or a secure application role, then you can grant the role indirectly to the user. An indirectly granted role is a role granted to the user through another role that has already been granted to this user. For example, suppose you grant user psmith the role1 role. Then you grant the role2 and role3 roles to the role1 role. Roles role2 and role3 are now under role1. This means psmith has been indirectly granted the roles role2 and role3, in addition to the direct grant of role1. Enabling the direct role1 for psmith enables the indirect roles role2 and role3 for this user as well.
- ☞ Optionally, you can make a directly granted role a default role. You enable or disable the default role status of a directly granted role by using the DEFAULT ROLE clause of the ALTER USER statement. Ensure that the DEFAULT ROLE clause refers only to roles that have been directly granted to the user. To find the directly granted roles for a user, query the DBA_ROLE_PRIVS data dictionary view. This view does not include the user's indirectly granted roles. To find roles that are granted to other roles, query the ROLE_ROLE_PRIVS view.
- ☞ If the role is password authenticated or a secure application role, then you cannot grant it indirectly to the user, nor can you make it a default role. You only can grant this type of role directly to the user. Typically, you enable password authenticated or secure application roles by using the SET ROLE statement.

Properties of Roles

- Roles have special properties that make their management very easy, such reduced privilege administration.

- Table below describes the properties of roles that enable easier privilege management within a database.

Property	Description
Reduced privilege administration	Rather than granting the same set of privileges explicitly to several users, you can grant the privileges for a group of related users to a role, and then only the role must be granted to each member of the group.
Dynamic privilege management	If the privileges of a group must change, then only the privileges of the role need to be modified. The security domains of all users granted the group's role automatically reflect the changes made to the role.
Selective availability of privileges	You can selectively enable or disable the roles granted to a user. This allows specific control of a user's privileges in any given situation.
Application awareness	The data dictionary records which roles exist, so you can design applications to query the dictionary and automatically enable (or disable) selective roles when a user attempts to execute the application by way of a given user name.
Application-specific security	You can protect role use with a password. Applications can be created specifically to enable a role when supplied the correct password. Users cannot enable the role if they do not know the password.

Oracle Supplied Roles (Predefined Roles)

- ☞ These roles are created automatically during database installation and serve specific purposes in managing database access and privileges. Here are some of the most commonly used Oracle-supplied roles:

DBA (Database Administrator):

The DBA role provides full database administrative privileges. Users granted this role have complete control over the database, including the ability to perform administrative tasks such as creating users, managing tablespaces, and executing system-level commands.

Provides a large number of system privileges, including the ANY privileges (such as the DELETE ANY TABLE and GRANT ANY PRIVILEGE privileges).

RESOURCE:

The RESOURCE role provides basic privileges necessary for developers to create and manage database objects. Users granted this role have permissions to create tables, procedures, sequences, and other objects in their schema. However, they do not have administrative privileges.

Provides the following system privileges:

CREATE CLUSTER,

CREATE INDEXTYPE,

CREATE OPERATOR,

CREATE PROCEDURE,

CREATE SEQUENCE,

CREATE TABLE,

CREATE TRIGGER,

CREATE TYPE.

CONNECT:

The CONNECT role provides basic privileges for users to connect to the database. Users granted this role can establish database sessions but have limited permissions beyond that. This role is often granted by default to new users.

Provides the CREATE SESSION system privilege.

CDB_DBA:

Provides the privileges required for administering a CDB, such as SET CONTAINER, SELECT ON PDB_PLUG_IN_VIOLATIONS, and SELECT ON CDB_LOCAL_ADMIN_PRIVS. If your site requires additional privileges, then you can create a role (either common or local) to cover these privileges, and then grant this role to the CDB_DBA role.

SELECT_CATALOG_ROLE:

The SELECT_CATALOG_ROLE role grants read-only access to data dictionary views. Users with this role can query metadata about database objects, users, privileges, and other system information.

EXECUTE_CATALOG_ROLE:

The EXECUTE_CATALOG_ROLE role grants execute privileges on certain data dictionary packages. Users with this role can execute procedures and functions in the SYS schema for accessing data dictionary information.

PDB_DBA:

Granted automatically to the local user that is created when you create a new PDB from the seed PDB. No privileges are provided with this role.

Creating Modifying and Dropping Roles

Creating Roles

- ☞ Use the CREATE ROLE statement to create a role, which is a set of privileges that can be granted to users or to other roles.
- ☞ You can add privileges to a role and then grant the role to a user. The user can then enable the role and exercise the privileges granted by the role.
- ☞ A role contains all privileges granted to the role and all privileges of other roles granted to it. A new role is initially empty. You add privileges to a role with the GRANT statement.
- ☞ If you create a role that is NOT IDENTIFIED or is IDENTIFIED EXTERNALLY or BY password, then Oracle Database grants

you the role with ADMIN OPTION. However, if you create a role IDENTIFIED GLOBALLY, then the database does not grant you the role. A global role cannot be granted to a user or role directly. Global roles can be granted only through enterprise roles.

- ☞ The **CREATE ROLE** statement in Oracle is used to create a new role within the database. Below is the syntax of the CREATE ROLE statement along with explanations of its various options:

```
CREATE ROLE role
  [ NOT IDENTIFIED
    | IDENTIFIED { BY password
                  | USING [ schema. ] package
                  | EXTERNALLY
                  | GLOBALLY AS '{ domain_name_of directory_group
  | ( AZURE_ROLE=value ) | ( IAM_GROUP_NAME=value ) }'
    }
  ] [ CONTAINER = { CURRENT | ALL } ] ;
```

Explanation of the options:

- ☞ *role*: Specifies the name of the role you want to create.
- ☞ *NOT IDENTIFIED*: This option indicates that the role does not require authentication. Roles created with this option are typically used for grouping privileges rather than for authentication purposes.
- ☞ *IDENTIFIED BY password*: Specifies that the role is identified by a password. Users granted this role must provide the password to use the privileges associated with the role.
- ☞ *IDENTIFIED USING [schema.] package*: Specifies that the role is identified by a PL/SQL package. The package specified can be used for custom authentication logic.
- ☞ *IDENTIFIED EXTERNALLY*: Specifies that the role is externally identified, meaning that the operating system or an external service is responsible for authenticating users assigned to this role.
- ☞ *GLOBALLY AS '{ domain_name_of directory_group | (AZURE_ROLE=value) | (IAM_GROUP_NAME=value) }'*: Specifies that the role is globally identified, indicating a role from an external identity provider such as Active Directory, Azure Active Directory, or Oracle Identity Cloud Service. You provide the domain name of the directory group, or specify the Azure role or IAM group name.

- ☞ `CONTAINER = { CURRENT | ALL }`: Specifies whether the role is created within the current container or all containers. This option is relevant for multitenant databases where you can create roles at the root level (ALL) or within a specific pluggable database (CURRENT).

Examples

```
CREATE ROLE dw_manager;
```

This statement creates the role `dw_manager`:

```
CREATE ROLE data_analyst IDENTIFIED BY "secure_password";
```

This creates a role named `data_analyst` and assigns it a password "secure_password". Users assigned this role will need to provide this password to activate the role and access its privileges.

```
CREATE ROLE warehouse_user IDENTIFIED GLOBALLY;
```

This statement creates global role `warehouse_user`.

```
CREATE ROLE role1 CONTAINER = CURRENT;
```

This statement creates local role `role1` in the current PDB. The current container must be a PDB when you issue this statement:

Modifying Roles

- ☞ In Oracle Database, you can modify roles using the `ALTER ROLE` statement. This statement allows you to add or remove privileges from a role, change its identification method, rename the role, or change other properties.
- ☞ You must either have been granted the role with the `ADMIN OPTION` or have `ALTER ANY ROLE` system privilege in order to modify the roles.
- ☞ Before you alter a role to `IDENTIFIED GLOBALLY`, you must:
 - Revoke all grants of roles identified externally to the role and
 - Revoke the grant of the role from all users, roles, and `PUBLIC`.

The one exception to this rule is that you should not revoke the role from the user who is currently altering the role.

- ☞ To specify the CONTAINER clause, you must be connected to a multitenant container database (CDB). To specify CONTAINER = ALL, the current container must be the root. To specify CONTAINER = CURRENT, the current container must be a pluggable database (PDB)

```
ALTER ROLE role
{ NOT IDENTIFIED
| IDENTIFIED
    { BY password
    | USING [ schema. ] package
    | EXTERNALLY
    | GLOBALLY AS '{domain_name_of_directory_group | (
AZURE_ROLE=value ) | ( IAM_GROUP_NAME=value )}'
    }
}
[ CONTAINER = { CURRENT | ALL } ] ;
```

The keywords, parameters, and clauses in the ALTER ROLE statement all have the same meaning as in the CREATE ROLE statement.

- ☞ You cannot alter a NOT IDENTIFIED role to any of the IDENTIFIED types if it is granted to another role.

Examples

The following statement changes the role warehouse_user (created in "Creating a Role: Example") to NOT IDENTIFIED:

```
ALTER ROLE warehouse_user NOT IDENTIFIED;
```

Following statement changes the password on the dw_manager role (created in "Creating a Role: Example") to data:

```
ALTER ROLE dw_manager IDENTIFIED BY data;
```

Dropping Roles

- ☞ Use the DROP ROLE statement to remove a role from the database. When you drop a role, Oracle Database revokes it from all users and roles to whom it has been granted and removes it from the database. User sessions in which the role is already enabled are not affected. However, no new user session can enable the role after it is dropped.
- ☞ You must have been granted the role with the ADMIN OPTION or you must have the DROP ANY ROLE system privilege to drop the roles
- ☞ Syntax:

DROP ROLE role;

Here *role* Specify the name of the role to be dropped.

Examples

To drop the role dw_manager, which was created in "Creating a Role: Example", issue the following statement:

DROP ROLE dw_manager;

Grant and Revoke Commands

- ☞ In Oracle Database, the GRANT and REVOKE statements are used to manage permissions and privileges granted to users or roles

GRANT Command

- ☞ Use the **GRANT** statement to grant:
 - System privileges to users and role
 - Roles to users, roles, and program units
 - Object privileges for a particular object to users and roles.
- ☞ To grant a **system privilege**, one of the following conditions must be met:
 - You must have been granted the GRANT ANY PRIVILEGE system privilege. In this case, if you grant the system privilege to a role, then a user to whom the role has been granted does not have the privilege unless the role is enabled in user's session.

- You must have been granted the system privilege with the ADMIN OPTION.
In this case, if you grant the system privilege to a role, then a user to whom the role has been granted has the privilege regardless whether the role is enabled in the user's session.
- ☞ To grant a **role to a user or another role**, you must have been directly granted the role with the ADMIN OPTION, or you must have been granted the GRANT ANY ROLE system privilege, or you must have created the role.
- ☞ To grant a **role to a program unit in your own schema**, you must have been directly granted the role with either the ADMIN OPTION or the DELEGATE OPTION, or you must have been granted the GRANT ANY ROLE system privilege, or you must have created the role.
- ☞ To **grant a role to a program unit in another user's schema**, you must be the user SYS and the role must have been created by the schema owner or directly granted to the schema owner.
- ☞ To grant **an object privilege on a user**, by specifying the ON USER clause of the on_object_clause, you must be the user on whom the privilege is granted, or you must have been granted the object privilege on that user with the WITH GRANT OPTION, or you must have been granted the GRANT ANY OBJECT PRIVILEGE system privilege.
- ☞ To grant an **object privilege on all other types of objects**, you must own the object, or the owner of the object must have granted you the object privileges with the WITH GRANT OPTION, or you must have been granted the GRANT ANY OBJECT PRIVILEGE system privilege.
- ☞ To specify the CONTAINER clause, you must be connected to a multitenant container database (CDB). To specify CONTAINER = ALL, the current container must be the root.

Granting system privilege

Syntax for granting system privilege:

```
GRANT {system_privilege} [ { WITH GRANT OPTION } ]
TO {username | PUBLIC | role_name}
[ CONTAINER = { CURRENT | ALL } ];
```

Explanation of the above syntax:

{system_privilege}: This specifies the system privilege you want to grant. Refer to Oracle documentation for a complete list of available system privileges. Some common examples include:

- **CREATE USER**: Allows creating new user accounts.
- **DROP USER**: Allows deleting user accounts.
- **CONNECT**: Allows users to connect to the database.
- **GRANT ANY**: Allows granting any privilege to users or roles.
- **ALTER SYSTEM**: Allows modifying various database settings.

[WITH GRANT OPTION]: This optional clause allows the grantee (user or role receiving the privilege) to further grant the same system privilege to other users. Use this with caution as it can lead to complex privilege hierarchies.

TO {username | PUBLIC | role_name}: This defines who receives the privilege:

- *username*: Grants the privilege to a specific user.
- *PUBLIC*: Grants the privilege to all users, including those not yet created.
- *role_name*: Grants the privilege to a role, which can then be assigned to multiple users.

[CONTAINER = { CURRENT | ALL }]: This optional clause is only relevant for certain system privileges. It defines the container (schema or database) in which the privilege applies:

- *CURRENT*: Grants the privilege within the current schema of the user issuing the GRANT statement (default if not specified).
- *ALL*: Grants the privilege within all schemas in the database.

Notes:

- ☞ You need sufficient privileges (usually administrative privileges) to grant system privileges.
- ☞ Be very cautious when granting system privileges, especially GRANT ANY. Only grant the minimum privileges required for users to perform their tasks.
- ☞ Review and manage system privileges regularly to ensure security and prevent unauthorized actions.

Example:

```
GRANT CREATE USER TO admin_user;
```

- This grants the CREATE USER privilege to the user admin_user, allowing them to create new user accounts within their current schema.

```
GRANT CREATE TABLE TO john;
```

- This allows the user john to create the table,

Granting object privilege

Syntax for granting system privilege:

```
GRANT {privilege_name} ON {object_name} TO {username | PUBLIC | role_name}
[ WITH GRANT OPTION ];
```

Explanation of the syntax :

{privilege_name}: This specifies the object privilege you want to grant. Common object privileges include:

- SELECT: Allows retrieving data from a table or view.
- INSERT: Allows inserting new data into a table.
- UPDATE: Allows modifying existing data in a table.
- DELETE: Allows deleting data from a table.
- CREATE TABLE: Allows creating new tables.

{object_name}: This refers to the specific database object on which the privilege is granted. This could be a table name, view name, sequence name, etc.

TO {username | PUBLIC | role_name}: This defines who receives the privilege:

- *username*: Grants the privilege to a specific user.
- *PUBLIC*: Grants the privilege to all users, including those not yet created (use with caution).
- *role_name*: Grants the privilege to a role, which can then be assigned to multiple users.

[WITH GRANT OPTION]: This optional clause allows the grantee (user or role receiving the privilege) to further grant the same object privilege to other users. Use this with caution as it can lead to complex privilege hierarchies.

Notes:

- The specific object privileges available depend on the type of object you're granting access to.
- You need appropriate privileges on the object itself to grant object privileges to others. (e.g., To grant SELECT on a table, you typically need SELECT or higher privilege on that table).
- Grant only the minimum privileges required for users or roles to perform their tasks.
- Review and manage object privileges regularly to ensure security and data integrity.

Examples:

- ☞ The Following example grants the user *user_name* the ability to update rows in the table named *table_name*

```
GRANT UPDATE ON table_name TO user_name;
```

- ☞ The following example grants the SELECT object privileges to a data warehouse user role, which was created in the "Creating a Role: Example":

```
GRANT SELECT ON sh.sales TO warehouse_user;
```

- ☞ The following statement grants the *warehouse_user* role to the *dw_manager* role. Both roles were created in the "Creating a Role: Example":

```
GRANT warehouse_user TO dw_manager;
```

The dw_manager role now contains all of the privileges in the domain of the warehouse_user role.

- ☞ To grant the INHERIT PRIVILEGES object privilege on user sh to user hr, issue the following statement:

```
GRANT INHERIT PRIVILEGES ON USER sh TO hr;
```

- ☞ To grant SELECT and UPDATE privileges on the view emp_view to all users, issue the following statement:

```
GRANT SELECT, UPDATE ON emp_view TO PUBLIC;
```

Granting roles to programs

- The syntax for granting roles to programs (typically stored procedures, functions, or packages) in Oracle uses the GRANT command with a specific clause

```
GRANT role_name TO program_name;
```

Explanation of the syntax:

- *role_name:* This refers to the existing role you want to grant to the program. This role likely contains a collection of object privileges that the program needs to execute its functionality.
- *program_name:* This specifies the name of the program to which you're granting the role. This can be the name of a stored procedure, function, or package.

Notes:

- You need to have the GRANT ANY ROLE system privilege or be the owner of the role to grant it to a program.
- Be cautious when granting roles to programs, as it can give them access to objects beyond the owner's permissions.
- Ensure proper privilege management for roles to maintain database security.

Examples

```
GRANT hr_role TO update_salary_proc;
```

This grants the role named `hr_role` to the stored procedure named `update_salary_proc`. Assuming the `hr_role` has the necessary privileges (e.g., `UPDATE` on the `salaries` table), the procedure can now update employee salary information.

Revoke Command

- Use the `REVOKE` statement to:
 - Revoke system privileges from users and roles
 - Revoke roles from users, roles, and program units.
 - Revoke object privileges for a particular object from users and roles
- To **revoke a system privilege**, you must have been granted the privilege with the `ADMIN OPTION`. You can revoke any privilege if you have the `GRANT ANY PRIVILEGE` system privilege.
- To **revoke a role from a user or another role**, you must have been directly granted the role with the `ADMIN OPTION` or you must have created the role. You can revoke any role if you have the `GRANT ANY ROLE` system privilege.
- To **revoke a role from a program unit**, you must be the user `SYS` or you must be the schema owner of the program unit.
- To revoke an object privilege, one of the following conditions must be met:
 - You must previously have granted the object privilege to the user or role.
 - You must have the `GRANT ANY OBJECT PRIVILEGE` system privilege.
In this case, you can revoke any object privilege that was granted by the object owner or on behalf of the owner by a user with the `GRANT ANY OBJECT PRIVILEGE`. However, you cannot revoke an object privilege that was granted by way of a `WITH GRANT OPTION` grant.
- The `REVOKE` statement can revoke only privileges and roles that were previously granted directly with a `GRANT` statement. You cannot use this statement to revoke:
 - Privileges or roles not granted to the revokee
 - Roles or object privileges granted through the operating system
 - Privileges or roles granted to the revokee through roles

- To specify the CONTAINER clause, you must be connected to a multitenant container database (CDB). To specify CONTAINER = ALL, the current container must be the root.

Revoking system privilege

Revoking with GRANT ANY PRIVILEGE

- If you have the GRANT ANY PRIVILEGE system privilege, you can revoke any system privilege using a simple syntax:

REVOKE privilege_name FROM username;

Here,

privilege_name: This is the name of the system privilege you want to revoke (e.g., CREATE USER, DROP TABLE).

username: This is the username from whom you want to revoke the privilege.

Revoking with ADMIN OPTION

- If you don't have GRANT ANY PRIVILEGE, you can still revoke a system privilege under specific conditions:
- *You granted the privilege with the ADMIN OPTION*: In this case, you can revoke the privilege regardless of whether the grantee was granted the GRANT OPTION.

REVOKE privilege_name FROM username;

- *The privilege was granted without the GRANT OPTION*: You can revoke the privilege only if it wasn't subsequently granted to others with the GRANT OPTION.

REVOKE privilege_name FROM username;

Examples

- The following statement revokes the DROP ANY TABLE system privilege from the users hr and oe:

```
REVOKE DROP ANY TABLE    FROM hr, oe;
```

The users hr and oe can no longer drop tables in schemas other than their own.

- The following statement revokes the role dw_manager from the user sh:

```
REVOKE dw_manager  FROM sh;
```

The user sh can no longer enable the dw_manager role.

- The following statement revokes the CREATE TABLESPACE system privilege from the dw_manager role:

```
REVOKE CREATE TABLESPACE  FROM dw_manager;
```

Enabling the dw_manager role no longer allows users to create tablespaces.

- To revoke the role dw_user from the role dw_manager, issue the following statement:

```
REVOKE dw_user  FROM dw_manager;
```

The dw_user role privileges are no longer granted to dw_manager.

Revoking object privilege

- The syntax for revoking object privileges in Oracle uses the REVOKE statement as follows

```
REVOKE privilege_name ON object_name FROM grantee [CASCADE CONSTRAINTS];
```

Where

- *REVOKE*: Keyword to initiate revoking privileges.
- *privilege_name*: Specific permission you want to revoke (e.g., SELECT, INSERT, UPDATE, DELETE, etc.).

- *ON object_name*: Specifies the database object on which the privilege is being revoked (e.g., table name, view name, etc.).
- *FROM grantee*: Username or role from whom the privilege is being revoked.
- *CASCADE CONSTRAINTS (Optional)*: If the revoked privilege is referenced in constraints, automatically removes those constraints as well.

Examples

- You can grant DELETE, INSERT, READ, SELECT, and UPDATE privileges on the table orders to the user hr with the following statement:

```
GRANT ALL ON orders TO hr;
```

To revoke the DELETE privilege on orders from hr, issue the following statement:

```
REVOKE DELETE ON orders FROM hr;
```

- To revoke the remaining privileges on orders that you granted to hr, issue the following statement:

```
REVOKE ALL ON orders FROM hr;
```

- Revoking SELECT privilege on the customers table from the sales role and also removing any constraints referencing it:

```
REVOKE SELECT ON customers FROM sales CASCADE CONSTRAINTS;
```

Using Predefined Roles

- Predefined roles in Oracle offer a convenient way to manage user privileges within your database. They group together commonly used system and object privileges, allowing you to assign them to users based on their functionalities and security needs.

How to Use Predefined Roles?

- *Identify User Needs:* Analyze the tasks and functionalities each user requires within the database.
- *Choose Appropriate Roles:* Select the pre-defined roles that best match the user's needs and security requirements. You can assign multiple roles to a user if necessary.
- *Grant Roles to Users:* Use the GRANT statement to assign the chosen roles to the respective users.